

NeuroLog AI: live logging guide

A practical guide to the repaired local application

1. Start both applications

From the project folder in PowerShell:

```
powershell -ExecutionPolicy Bypass -File .\start-preview.ps1
```

Python 3.12 and Node.js must be installed. The launcher creates a Python environment, installs missing dependencies, and starts services in the background. Open the task application at <http://127.0.0.1:8000> and NeuroLog at <http://127.0.0.1:5173/dashboard>. No login is required.

After changing backend code, restart managed services using the same command with `-Restart`. Services started outside the launcher must be stopped separately if they occupy these ports.

2. Generate real application activity

1. Enter a task title and click Add task.
2. Click Complete, or Reopen, on that task.
3. Open NeuroLog's Log Explorer and look for source task-service.
4. Use search and severity filters to inspect the received events.
5. Click Run clustering, then open Vector Analysis to inspect message groups and outliers.

Task changes are persisted in SQLite. Each request produces a log containing method, path, status and measured duration. Every five seconds, a heartbeat records measured uptime, request count, tasks, completions and queued events. Healthy activity normally produces INFO logs; genuine rejected requests produce WARNING logs and failures produce ERROR logs. No events are preloaded. Optional random generation is available through Start generating and Stop generating, with a configurable interval (0.2 to 10 seconds) and entry limit (1 to 1000). Each generated event starts with [GENERATED], and the run stops at its limit.

The dashboard polls every two seconds and Explorer every three seconds, so delivery and display are near real time rather than a push connection. Clustering runs when requested; dashboard updates alone do not retrain it.

3. How the connection works

Python application -> logging.Handler -> durable SQLite outbox -> HTTP POST /api/ingest -> NeuroLog SQLite -> dashboard polling.

`live_app/app.py` implements the task service. `live_app/log_shipper.py` implements the reusable NeuroLogHandler. A worker sends queued events asynchronously, using an ingestion key obtained automatically from the local API. The key is cached in `data/task-service.key` and excluded from Git.

The sender removes an event only after a successful response. If NeuroLog is unavailable, it retries with backoff up to 30 seconds. Queued events survive restarts. Delivery is at least once: an interrupted acknowledgment can produce a duplicate. The outbox holds at most 10,000 events; once full it reports the dropped new event to stderr.

Python levels map to syslog severity: CRITICAL=2, ERROR=3, WARNING=4, INFO=6, DEBUG=7. Request headers, query strings and task titles are excluded from routine request logs. Avoid adding credentials or private data to your own log messages.

4. Connect another Python application

Import the handler from this project, create it once at startup, and attach it to your application's logger:

```
import logging
from live_app.log_shipper import NeuroLogHandler

handler = NeuroLogHandler(
    endpoint='http://127.0.0.1:5001/api/ingest',
    source='my-python-service',
    spool_path='data/my-service-outbox.db',
    key_path='data/my-service.key',
)
handler.start()
logger = logging.getLogger('my-python-service')
logger.setLevel(logging.INFO)
logger.addHandler(handler)
# In actual application operations:
logger.info('Order processed order_id=%s', order_id)
# At shutdown:
handler.close()
```

The `order_id` above comes from your application's real operation. Use a separate source, key file and outbox per application. The sample task service already implements this pattern, so it requires no additional code to run.

To run only the task application, activate the project's environment and execute `python -m live_app.app`. It reads `NEUROLOG_API_URL` and `NEUROLOG_INGEST_KEY` from environment variables. For a remote endpoint, supply its ingestion key explicitly; automatic key provisioning is restricted to localhost.

5. Data and analysis

Received records use `data/neurolog-live.db`; tasks use `data/tasks.db`; undelivered events use `data/task-log-outbox.db`. Older records in `data/neurolog.db` remain preserved but are not loaded into the current workspace. The `data` folder is ignored by Git.

TF-IDF transforms received message text into vectors. DBSCAN groups similar vectors and marks noise as candidate outliers. Review unusual messages in context. The health score is a heuristic derived from recent critical logs, not a verified uptime measurement. The task service heartbeat contains actual measured uptime.

Without `GROQ_API_KEY`, the assistant groups received warnings, errors and stored outliers using transparent local rules, cites evidence IDs, and suggests investigation steps. Enter `source:task-service` to narrow the local analysis. Generated and actual incidents are kept separate. The interface shows the active mode; provider failure falls back to local analysis with a notice. With a valid optional Groq key configured for

the backend, it requests conversational analysis. It never executes fixes.

6. Troubleshooting and validation

If startup fails, inspect `data/backend.stderr.log`, `data/tasks.stderr.log`, and `data/frontend.stderr.log`. A port conflict requires stopping the conflicting service. Do not start multiple task-service processes against the same outbox.

If messages stop appearing, check `http://127.0.0.1:5001/api/health` and `http://127.0.0.1:8000/health`. Delivery warnings are written to the task-service stderr log. Restore the backend; the sender retries automatically. After a restart, allow up to 30 seconds for backoff recovery, plus the dashboard polling interval.

Run `.venv/Scripts/python.exe -m pytest -q` for ingestion, isolation, clustering, task operations and queue-recovery checks. Run `npm run lint` and `npm run build` inside `loginTEL_ui` for frontend validation.

The login-free local workspace binds to localhost. A shared deployment needs proper service hosting and an authenticated interface; do not expose the current local workspace publicly. Original project: <https://github.com/Sai-Nitin123/Neurolog-AI>; MIT attribution is retained.